ASSIGNMENT NO.1

Question 1 — IoT Sensor Dashboard

Problem 1 — Sensor Reading Classifier

You have joined a team building firmware for an industrial factory monitoring system. The factory floor has temperature sensors installed near heavy machinery. Each sensor sends a single double reading (in °C) to the control unit every few seconds.

Your task is to write the classification module — the piece of code that reads one sensor value, maps it to a status, and prints what action the control unit should take.

Range (°C)	Status Code	Status Label	Action
< 0	-1	SENSOR_ERROR	Sensor fault — check wiring
0 – 29	0	NORMAL	No action required
30 – 44	1	WARNING	Alert sent to supervisor
45 – 59	2	CRITICAL	Cooling system triggered
≥ 60	3	SHUTDOWN	Emergency shutdown initiated

Requirement:

- Store the reading as double, derive a status code as int using if-else
- Use switchon the status code to print the action
- Use the **ternary operator** to print Above Averageor Below Averagerelative to 25°C (normal operating temperature)
- Print the temperature in Fahrenheit as well: F = (C × 9 / 5) + 32

Code:

```
#include <iostream>
using namespace std;
int main() {
   double tempC;
   cout << "Enter sensor temperature reading (C): ";
   cin >> tempC;

   int statusCode;
   if (tempC < 0) {
      statusCode = -1;
```

```
    } else if (tempC <= 29) {
        statusCode = 0;
    } else if (tempC <= 44) {
        statusCode = 1;
    } else if (tempC <= 59) {
        statusCode = 2;
    } else {
        statusCode = 3;
    }
    cout << "\nTemperature: " << tempC << " C" << endl;
    switch (statusCode) {
        case -1:
            cout << "Status : SENSOR_ERROR" << endl;
            cout << "Action: Sensor fault - check wiring" << endl;
            break;
        case 0:
            cout << "Status : NORMAL" << endl;
            cout << "Action: No action required" << endl;
            break;
        case 1:
            cout << "Status : WARNING" << endl;
            cout << "Action: Alert sent to supervisor" << endl;
            break;
        case 2:
            cout << "Status : CRITICAL" << endl;
            cout << "Action: Cooling system triggered" << endl;
            break;
        case 3:
            cout << "Status : SHUTDOWN" << endl;
            cout << "Action: Emergency shutdown initiated" << endl;
            break;
    }
    string comparison = (tempC > 25) ? "Above Average" : "Below Average";
    cout << "Triggered Reading : " << comparison << endl;

    double tempF = (tempC * 9 / 5) + 32;
    cout << "Temperature in Fahrenheit: " << tempF << " F" << endl;
    return 0;
}
```

Expected Output:
Enter sensor temperature reading (C): 47.3
Temperature: 47.3 C
Status : CRITICAL
Action: Cooling system triggered
Triggered Reading : Above Average
Temperature in Fahrenheit: 117.14 F

Problem 2 — Sensor Log Buffer

The factory sensors now send one reading per second. Your senior engineer tells you:
"We can't store everything — just keep the last N readings in a buffer and run basic analysis on them."

You are asked to write the buffer analysis module. It stores readings in a fixed-size array and generates a status report.

Requirements:
1.Accept N from the user ($1 \leq N \leq 100$), then read N temperature values into an array
2.Print all valid readings — skip values below 0 (sensor error) using continue
3.Scan for the first reading at or above 45°C — print its index and stop scanning using break
4.Compute min, max, and average in one single loop pass
5.Count readings per category: Normal / Warning / Critical / Shutdown.

Code:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
   int n;
   cout << "Enter number of readings (1-100): ";
   cin >> n;


   while (n < 1 || n > 100) {
      cout << "Invalid! Enter N between 1 and 100: ";
      cin >> n;
   }

   double readings[100];
```

```
for (int i = 0; i < n; i++) {
   cout << "Enter reading " << i << ": ";
   cin >> readings[i];
}


int skippedCount = 0;
int validCount   = 0;

double sum = 0;
double minVal = 1e9;
double maxVal = -1e9;

int normalCount = 0, warningCount = 0, criticalCount = 0, shutdownCount = 0;
cout << "\nValid readings : ";
for (int i = 0; i < n; i++) {
   double temp = readings[i];


   if (temp < 0) {
      skippedCount++;
      continue;
   }

   validCount++;
   cout << fixed << setprecision(1) << temp << " ";

   sum += temp;

   if (temp < minVal) minVal = temp;
   if (temp > maxVal) maxVal = temp;

   if (temp >= 0 && temp <= 29) {
      normalCount++;
   }
   else if (temp >= 30 && temp <= 44) {
      warningCount++;
   }
   else if (temp >= 45 && temp <= 59) {
      criticalCount++;
   }
   else if (temp >= 60) {
      shutdownCount++;
   }
```

```
    }
    cout << "\nSkipped (errors) : " << skippedCount << endl;

    bool found = false;
    for (int i = 0; i < n; i++) {
        if (readings[i] >= 45) {
            cout << "First CRITICAL : Index " << i << " -> "
                 << fixed << setprecision(1) << readings[i] << "C" << endl;
            found = true;
            break;
        }
    }
    if (!found) {
        cout << "First CRITICAL : None found" << endl;
    }
    double avg = sum / validCount;
    cout << "Min : " << fixed << setprecision(1) << minVal << "C  "
         << "Max : " << maxVal << "C  "
         << "Avg : " << setprecision(2) << avg << "C" << endl;

    cout << "Normal:" << normalCount
         << "  Warning:" << warningCount
         << "  Critical:" << criticalCount
         << "  Shutdown:" << shutdownCount << endl;

    return 0;
}
```

Output:

```
Enter number of readings (1-100): 8
Enter reading 0: 22.1
Enter reading 1: 31.5
Enter reading 2: -5
Enter reading 3: 46.0
Enter reading 4: 28.0
Enter reading 5: 50.2
Enter reading 6: 10.0
Enter reading 7: 38.0

Valid readings : 22.1 31.5 46.0 28.0 50.2 10.0 38.0
Skipped (errors) : 1
First CRITICAL : Index 3 -> 46.0C
Min : 10.0C  Max : 50.2C  Avg : 32.26C
Normal:3  Warning:2  Critical:2  Shutdown:0
```

Problem 3 — Building Sensor Grid

The system has been scaled to a full smart building. Three floors, three rooms per floor — a 3×3 grid of sensors. The building manager wants a floor-by-floor status report on his dashboard every morning.
You are assigned to write the grid reading and reporting module.

Requirements:
1.Read temperatures for all 9 rooms into a 2D array (rows = floors, columns = rooms)
2.Display the readings in a formatted table
3.Find and report the hottest room (floor and room number)
4.Find and report the floor with the highest average temperature
5.Count total rooms at or above the WARNING threshold (830°C)

Code:

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
   const int floors = 3;
   const int rooms  = 3;
   const double WARNING_THRESHOLD = 30.0;

   double grid[floors][rooms];

   for (int f = 0; f < floors; f++) {
      for (int r = 0; r < rooms; r++) {
         cout << "Enter temp for Floor " << (f + 1)
              << " Room " << (r + 1) << ": ";
         cin >> grid[f][r];
      }
   }

   cout << "\n";
   cout << "          " << setw(8) << "Room1" << setw(8) << "Room2" << setw(8) << "Room3" << endl;

   for (int f = 0; f < floors; f++) {
      cout << "Floor " << (f + 1) << " : ";
      for (int r = 0; r < rooms; r++) {
         cout << setw(8) << fixed << setprecision(1) << grid[f][r];
      }
      cout << endl;
```

```
}

double hottestTemp = grid[0][0];
int hottestFloor = 0, hottestRoom = 0;

double floorSum[floors] = {0, 0, 0};
int warningCount = 0;

for (int f = 0; f < floors; f++) {
   for (int r = 0; r < rooms; r++) {

      double temp = grid[f][r];

      floorSum[f] += temp;

      if (temp > hottestTemp) {
         hottestTemp = temp;
         hottestFloor = f;
         hottestRoom = r;
      }

      if (temp >= WARNING_THRESHOLD) {
         warningCount++;
      }
   }
}

double floorAvg[floors];
int hottestFloorIndex = 0;

for (int f = 0; f < floors; f++) {
   floorAvg[f] = floorSum[f] / rooms;
   if (floorAvg[f] > floorAvg[hottestFloorIndex]) {
      hottestFloorIndex = f;
   }
}
cout << "\nHottest Room    : Floor " << (hottestFloor + 1)
    << ", Room " << (hottestRoom + 1)
    << " -> " << fixed << setprecision(1) << hottestTemp << "C" << endl;

cout << "Hottest Floor   : Floor " << (hottestFloorIndex + 1)
    << "  (avg " << setprecision(2) << floorAvg[hottestFloorIndex] << "C)" << endl;

cout << "Rooms at WARNING or above : " << warningCount << endl;
```

```
    return 0;
}
```

Expected Output:

```
Enter temp for Floor 1 Room 1: 24.0
Enter temp for Floor 1 Room 2: 31.5
Enter temp for Floor 1 Room 3: 28.0
Enter temp for Floor 2 Room 1: 45.0
Enter temp for Floor 2 Room 2: 22.0
Enter temp for Floor 2 Room 3: 30.0
Enter temp for Floor 3 Room 1: 19.0
Enter temp for Floor 3 Room 2: 27.5
Enter temp for Floor 3 Room 3: 50.2

           Room1   Room2   Room3
Floor 1 :    24.0    31.5    28.0
Floor 2 :    45.0    22.0    30.0
Floor 3 :    19.0    27.5    50.2

Hottest Room    : Floor 3, Room 3 -> 50.2C
Hottest Floor   : Floor 2  (avg 32.33C)
Rooms at WARNING or above : 4
```

Problem 4 — Startup Configuration via CLI

The monitoring program now runs as a background daemon on an embedded Linux device. The operations team does not want to recompile the code every time thresholds change — they want to pass thresholds at startup via the command line, the same way they configure nginx or any other service.

You are asked to write the configurable version of the monitor.

Program name: sensor_monitor
Usage: ./sensor_monitor <warn_threshold> <critical_threshold> <num_readings>
Requirements:
- If arguments are missing, print a usage line and exit with code 1
- Validate: warn < critical, 1 ≤ num_readings ≤ 500 — print a specific error and exit on failure
- If valid, simulate num_readings temperature values using rand() % 70, classify each using the provided thresholds, and print a category summary

Code:

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main(int argc, char* argv[]) {

   if (argc < 4) {
      cout << "Usage   : ./sensor_monitor <warn_threshold> <critical_threshold> <num_readings>" <<
endl;
      cout << "Error   : Missing arguments." << endl;
      return 1;
   }

   int warnThreshold     = atoi(argv[1]);
   int criticalThreshold = atoi(argv[2]);
   int numReadings       = atoi(argv[3]);

   if (warnThreshold >= criticalThreshold) {
      cout << "Error   : warn_threshold must be less than critical_threshold." << endl;
      return 1;
   }

   if (numReadings < 1 || numReadings > 500) {
      cout << "Error   : num_readings must be between 1 and 500." << endl;
      return 1;
   }

   cout << "Config  : Warn=" << warnThreshold << "C  "
        << "Critical=" << criticalThreshold << "C  "
        << "Readings=" << numReadings << endl;

   int shutdownThreshold = criticalThreshold + (criticalThreshold - warnThreshold);
   int normalCount = 0, warningCount = 0, criticalCount = 0, shutdownCount = 0;

   for (int i = 0; i < numReadings; i++) {
      int temp = rand() % 70;
      if (temp < warnThreshold) {
         normalCount++;
      }
      else if (temp < criticalThreshold) {
         warningCount++;
      }
```

```
        else if (temp < shutdownThreshold) {
            criticalCount++;
        }
        else {
            shutdownCount++;
        }
    }
    cout << "Results : Normal:" << normalCount
         << "  Warning:" << warningCount
         << "  Critical:" << criticalCount
         << "  Shutdown:" << shutdownCount << endl;

    return 0;
}
```

Output:

```
D:\C++\named sensor_monitor.cpp\src>sensor_monitor.exe 30 45 10
Config  : Warn=30C  Critical=45C  Readings=10
Results : Normal:6  Warning:1  Critical:3  Shutdown:0
D:\C++\named sensor_monitor.cpp\src>sensor_monitor.exe 30
Usage   : ./sensor_monitor <warn_threshold> <critical_threshold> <num_readings>
Error   : Missing arguments.
```

Question 2 — Embedded Memory Manager & Signal Processor

Problem 1 — The Bug in Sensor Recalibration

Your team lead assigns you a code review task. A junior developer submitted a function resetSensorPair()
— it is supposed to swap two sensor readings before recalibration runs. During testing, the sensors are not swapping. The team lead says:
"Figure out why it doesn't work and give me two fixed versions — one using references, one using pointers. Then write a comment explaining the root cause. We'll use it in the next team knowledge-sharing session."

Write all three versions:
void resetSensorPairV1(int reading1, int reading2);
Value
void resetSensorPairV2(int& reading1, int& reading2);
Reference
void resetSensorPairV3(int* reading1, int* reading2);

Call all three from main(). Print values before and after each call. Write a comment block (minimum 4 lines) inside main() explaining why V1 fails.

Code:

```
#include <iostream>
using namespace std;

//  Call by Value
void resetSensorPairV1(int reading1, int reading2) {
   int temp = reading1;
   reading1 = reading2;
   reading2 = temp;
}

// Call by Reference
void resetSensorPairV2(int& reading1, int& reading2) {
   int temp = reading1;
   reading1 = reading2;
   reading2 = temp;
}

// Call by Pointer
void resetSensorPairV3(int* reading1, int* reading2) {
   int temp = *reading1;
   *reading1 = *reading2;
   *reading2 = temp;
}

int main() {

   int A = 55, B = 12;

cout << "--- V1: Call by Value ---" << endl;
cout << "Before Swap: A=" << A << "    B=" << B << endl;
resetSensorPairV1(A, B);
cout << "After Swap : A=" << A << "    B=" << B  << endl;

cout << "\n--- V2: Call by Reference ---" << endl;
cout << "Before Swap: A=" << A << "   B=" << B << endl;
resetSensorPairV2(A, B);
cout << "After  Swap: A=" << A << "   B=" << B <<  endl;

cout << "\n--- V3: Call by Pointer ---" << endl;
cout << "Before Swap: A=" << A << "   B=" << B << endl;
resetSensorPairV3(&A, &B);
cout << "After  Swap: A=" << A << "   B=" << B << endl;
```

```
    return 0;
}
```

Output:

```
--- V1: Call by Value ---
Before Swap: A=55    B=12
After Swap : A=55    B=12

--- V2: Call by Reference ---
Before Swap: A=55   B=12
After  Swap: A=12   B=55

--- V3: Call by Pointer ---
Before Swap: A=12   B=55
After  Swap: A=55   B=12
```

Problem 2 — Signal Processing Pipeline

You are building a pre-processing module for an audio analysis system. Raw audio data arrives as an array of double samples. Before any AI model can process it, the signal must go through a fixed pipeline of operations.

Your senior writes the function signatures and says:

"No array indexing inside these functions. Use pointer arithmetic — that's how the DSP library we're interfacing with operates."

```
double computeRMS(double* signal, int n);
// Returns sqrt( sum of (each element squared) / n )

void normalise(double* signal, int n);
// Divides every element by the max absolute value in the array (in-place)

int countZeroCrossings(double* signal, int n);
// Returns count of positions where adjacent elements have opposite signs

void applyGain(double* signal, int n, double gainFactor);
// Multiplies every element by gainFactor (in-place)
```

Rule: No arr[i] inside any of these functions. Use *ptr, *(ptr + i), or ptr++ only.

Test signal: {0.5, -1.2, 0.8, -0.3, 1.0, -0.9, 0.1}

Print the array before and after calling normalise() and applyGain(). Print the value returned by computeRMS() and countZeroCrossings().

Code:

```
#include <iostream>
#include <cmath>
#include <iomanip>
using namespace std;

void printArray(double* signal, int n) {
    double* p = signal;
    for (int i = 0; i < n; i++) {
        cout << fixed << setprecision(4) << *p << " ";
        p++;
    }
    cout << endl;
}

double computeRMS(double* signal, int n) {
    double sum = 0;
    double* p = signal;
    for (int i = 0; i < n; i++) {
        sum += (*p) * (*p);
        p++;
    }
    return sqrt(sum / n);
}

void normalise(double* signal, int n) {

    double maxAbs = 0;
    double* p = signal;
    for (int i = 0; i < n; i++) {
        double val = *p;
        if (val < 0) val = -val;
        if (val > maxAbs) maxAbs = val;
        p++;
    }

    p = signal;
    for (int i = 0; i < n; i++) {
        *p = *p / maxAbs;
        p++;
    }
}
```

```
int countZeroCrossings(double* signal, int n) {
    int count = 0;
    double* p = signal;
    for (int i = 0; i < n - 1; i++) {
        double curr = *p;
        double next = *(p + 1);
        if ((curr < 0 && next > 0) || (curr > 0 && next < 0)) {
            count++;
        }
        p++;
    }
    return count;
}

void applyGain(double* signal, int n, double gainFactor) {
    double* p = signal;
    for (int i = 0; i < n; i++) {
        *p = (*p) * gainFactor;
        p++;
    }
}

int main() {
    double signal[] = {0.5, -1.2, 0.8, -0.3, 1.0, -0.9, 0.1};
    int n = 7;

    cout << "Original signal   : ";
    printArray(signal, n);

    double rms = computeRMS(signal, n);
    int crossings = countZeroCrossings(signal, n);

    cout << "RMS               : " << fixed << setprecision(4) << rms << endl;
    cout << "Zero Crossings    : " << crossings << endl;

    normalise(signal, n);
    cout << "\nAfter normalise() : ";
    printArray(signal, n);

    double gainFactor = 2.0;
    applyGain(signal, n, gainFactor);
    cout << "After applyGain(" << gainFactor << ") : ";
    printArray(signal, n);
```

```
    return 0;
}
```

Output:

```
Original signal:
0.5000  -1.2000 0.8000  -0.3000 1.0000  -0.9000 0.1000

RMS value: 0.7783
Zero Crossings: 6

After normalise():
0.4167  -1.0000 0.6667  -0.2500 0.8333  -0.7500 0.0833

After applyGain(gain=2.0000):
0.8333  -2.0000 1.3333  -0.5000 1.6667  -1.5000 0.1667
```

Problem 3 — Hardware Register Access

You are writing low-level firmware that communicates with a sensor chip. The chip has three types of registers mapped to memory addresses:

- **Status register** — the firmware must only read this (the chip writes to it)
- **Control register** — the firmware writes to it, and it must always point to the same address
- **ROM config** — fixed value burned at factory, never changes

If a developer accidentally writes to the status register, the chip enters an undefined state. Your job is to enforce these access rules using const pointer types.

```
int statusReg   = 0b10110001;  // Read-only from firmware side
int controlReg = 0b00000000;// Firmware writes here
int dataReg      = 0b11001010;// For reassignment demo
```

Requirements:

- Declare regPtr1 as const int* pointing to statusReg — print its value; attempt a write and a repoint, comment both out with error explanation
- Declare regPtr2 as int* const pointing to controlReg — write a new value through it; attempt a repoint, comment it out with explanation
- Declare regPtr3 as const int* const pointing to statusReg — print its value; attempt both write and repoint, comment both out with explanations

Code:

```
#include <iostream>
using namespace std;

int main() {
   int statusReg  = 0b10110001;
   int controlReg = 0b00000000;
   int dataReg    = 0b11001010;

   const int* regPtr1 = &statusReg;
   cout << "regPtr1 (statusReg) = " << *regPtr1 << endl;

   int* const regPtr2 = &controlReg;
   *regPtr2 = 0b00000001;
   cout << "regPtr2 (controlReg) new value = " << *regPtr2 << endl;

   const int* const regPtr3 = &statusReg;
   cout << "regPtr3 (statusReg) = " << *regPtr3 << endl;

   return 0;
}
```

Output:

```
regPtr1 (statusReg) = 177
regPtr2 (controlReg) new value = 1
regPtr3 (statusReg) = 177
```

Problem 4 — Calibration Packet Parser

Your embedded device receives raw integer data packets over a serial bus. Before the main application can use the data, a parser must scan each packet and locate the minimum and maximum values for sensor calibration. These two values must be handed back to the caller.

The protocol team says: "Don't copy the values — give back pointers into the original packet buffer. We need to know their positions too."

```
bool parsePacket(const int* rawData, int size,
int** outMin, int** outMax);
```

main() — use exactly as given:

```
int packet[] = {45, 12, 67, 8, 55, 31};
int* minPtr = nullptr;
```

```
 int* maxPtr = nullptr;

if (parsePacket(packet, 6, &minPtr, &maxPtr)) {
cout << "Calibration Min : " << *minPtr << endl;
cout << "Calibration Max : " << *maxPtr << endl;
}
```

Code:

```
#include <iostream>
using namespace std;
bool parsePacket(const int* rawData, int size, int** outMin, int** outMax) {
    if (rawData == nullptr || size <= 0) {
        return false;
    }

    const int* p = rawData;
    const int* minPtr = rawData;
    const int* maxPtr = rawData;

    for (int i = 0; i < size; i++) {
        if (*p < *minPtr) {
            minPtr = p;
        }
        if (*p > *maxPtr) {
            maxPtr = p;
        }
        p++;
    }

    *outMin = const_cast<int*>(minPtr);
    *outMax = const_cast<int*>(maxPtr);

    return true;
}

int main() {
    int packet[] = {45, 12, 67, 8, 55, 31};
    int* minPtr = nullptr;
    int* maxPtr = nullptr;

    if (parsePacket(packet, 6, &minPtr, &maxPtr)) {
        cout << "Calibration Min : " << *minPtr << endl;
```

```
        cout << "Calibration Max : " << *maxPtr << endl;
    }
    return 0;
}
```

Expected Output:
Calibration Min : 8
Calibration Max : 67

Problem 5 — Drone Navigation Utilities

You are part of a team building the flight controller for an autonomous delivery drone. The navigation module needs a small set of math helper functions used hundreds of times per second. Your lead engineer says:
"Write these as inline functions. We don't want function call overhead in the navigation loop — the compiler should inline the code directly."

```
inline double distanceBetween(double x1, double y1, double x2, double y2);
// sqrt( pow(x2-x1, 2) + pow(y2-y1, 2) )
inline double toRadians(double degrees);
// degrees * (M_PI / 180.0)

inline double clamp(double value, double minVal, double maxVal);
// Restrict value to [minVal, maxVal]
inline bool isInSafeZone(double x, double y, double cx, double cy, double radius);
// true if point (x,y) is within the circle centred at (cx,cy) with given radius
```

In main(), set home position as (0.0, 0.0) and safe-zone radius as 50.0 units. Test with 3 waypoints of your choice. For each waypoint print its distance from home and whether it is inside the safe zone.

Code:

```
#include <iostream>
#include <cmath>
#include <iomanip>
using namespace std;

inline double distanceBetween(double x1, double y1, double x2, double y2) {
   return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
}

inline double toRadians(double degrees) {
   return degrees * (M_PI / 180.0);
}

inline double clamp(double value, double minVal, double maxVal) {
   if (value < minVal) return minVal;
   if (value > maxVal) return maxVal;
   return value;
}

inline bool isInSafeZone(double x, double y, double cx, double cy, double radius) {
   double d = distanceBetween(x, y, cx, cy);
   return d <= radius;
}

int main() {
   double homeX = 0.0, homeY = 0.0;
   double safeRadius = 50.0;

   double waypoints[3][2] = {
      {30.0, 20.0},
      {45.0, 45.0},
      {10.0, -70.0}
   };

   for (int i = 0; i < 3; i++) {
      double wx = waypoints[i][0];
      double wy = waypoints[i][1];

      double dist = distanceBetween(homeX, homeY, wx, wy);
      bool safe = isInSafeZone(wx, wy, homeX, homeY, safeRadius);

      cout << "Waypoint " << (i + 1) << " (" << wx << ", " << wy << ") "
```

```
        << "-> Distance: " << fixed << setprecision(2) << dist
        << "  SafeZone: " << (safe ? "YES" : "NO") << endl;
    }

    return 0;
}
```

Output:
Waypoint 1 (30, 20) -> Distance: 36.06 SafeZone: YES
Waypoint 2 (45.00, 45.00) -> Distance: 63.64 SafeZone: NO
Waypoint 3 (10.00, -70.00) -> Distance: 70.71 SafeZone: NO

Question 3 — HR Payroll Engine

The Scenario

Your company has won a contract to build an HR module for a mid-sized manufacturing firm. The existing system stores employee data directly in global variables — anyone in the codebase can accidentally set a salary to a negative number or assign an employee to a non-existent department, causing payroll errors at month end.

Your task is to redesign this as a proper C++ class that enforces data rules at the point of entry, so invalid data never enters the system in the first place.

Class: Employee
Private data members:

Member	Type	Rule
`empId`	`int`	Auto-assigned starting from 1001, increments per object created
`name`	`string`	Cannot be empty
`department`	`string`	Must be one of: Engineering, HR, Finance, Operations
`grade`	`char`	Only `'A'`, `'B'`, `'C'`, `'D'` accepted
`basicSalary`	`double`	Must be > 10,000 and < 5,00,000
`isActive`	`bool`	Default `true`; only `deactivate()` can set it to `false`
`employeeCount`	`static int`	Shared across all objects — counts total employees created

Code:

```
#include <iostream>
#include <iomanip>
#include <string>
#include <sstream>
using namespace std;

string formatMoney(double amount) {
    ostringstream out;
    out << fixed << setprecision(2) << amount;
    string s = out.str();

    size_t dotPos = s.find('.');
    string intPart = s.substr(0, dotPos);
    string decPart = s.substr(dotPos);

    string result;
    int count = 0;
    for (int i = intPart.size() - 1; i >= 0; i--) {
        result = intPart[i] + result;
        count++;
        if (count % 3 == 0 && i != 0) {
            result = "," + result;
        }
    }

    return result + decPart;
}

string centerText(const string& text, int width) {
    int totalPadding = width - (int)text.size();
    if (totalPadding <= 0) return text;
    int left = totalPadding / 2;
    int right = totalPadding - left;
    return string(left, ' ') + text + string(right, ' ');
}

class Employee {
private:
    int empId;
    string name;
    string department;
    char grade;
```

```
    double basicSalary;
    bool isActive;

    static int employeeCount;
    static int nextEmpId;

public:
    Employee() {
        empId = nextEmpId++;
        name = "";
        department = "";
        grade = ' ';
        basicSalary = 0.0;
        isActive = true;
        employeeCount++;
    }

    void setName(const string& n) {
        if (n.empty()) {
            cout << "ERROR: Name cannot be empty." << endl;
            return;
        }
        name = n;
    }

    void setDepartment(const string& dept) {
        if (dept == "Engineering" || dept == "HR" || dept == "Finance" || dept == "Operations") {
            department = dept;
        } else {
            cout << "ERROR: '" << dept << "' is not a registered department." << endl;
        }
    }

    void setGrade(char g) {
        if (g == 'A' || g == 'B' || g == 'C' || g == 'D') {
            grade = g;
        } else {
            cout << "ERROR: Invalid grade '" << g << "'. Accepted values: A, B, C, D." << endl;
        }
    }

    void setBasicSalary(double salary) {
        if (salary > 10000 && salary < 500000) {
            basicSalary = salary;
```

```
    } else {
        cout << "ERROR: Salary must be between Rs.10,000 and Rs.5,00,000. Value rejected." << endl;
    }
}

void deactivate() {
    isActive = false;
}

int getEmpId() const { return empId; }
string getName() const { return name; }
string getDepartment() const { return department; }
char getGrade() const { return grade; }
double getBasicSalary() const { return basicSalary; }
bool getIsActive() const { return isActive; }

double computeAllowances() const {
    switch (grade) {
        case 'A': return basicSalary * 0.40;
        case 'B': return basicSalary * 0.30;
        case 'C': return basicSalary * 0.20;
        case 'D': return basicSalary * 0.10;
        default:  return 0.0;
    }
}

double computeGrossSalary() const {
    return basicSalary + computeAllowances();
}

double computeTax() const {
    double gross = computeGrossSalary();
    if (gross <= 20000) return 0.0;
    else if (gross <= 50000) return gross * 0.05;
    else if (gross <= 100000) return gross * 0.10;
    else return gross * 0.20;
}

double computeNetSalary() const {
    return computeGrossSalary() - computeTax();
}

void printPayslip() const {
    int allowancePercent = 0;
```

```
        switch (grade) {
            case 'A': allowancePercent = 40; break;
            case 'B': allowancePercent = 30; break;
            case 'C': allowancePercent = 20; break;
            case 'D': allowancePercent = 10; break;
        }

        const int WIDTH = 46;
        string border(WIDTH, '=');
        string dash(WIDTH, '-');

        ostringstream allowLabel;
        allowLabel << "Allowances (" << allowancePercent << "%)";

        cout << border << endl;
        cout << centerText("EMPLOYEE PAYSLIP - AUG 2026", WIDTH) << endl;
        cout << border << endl;
        cout << left << setw(16) << "Emp ID"     << ": " << empId << endl;
        cout << left << setw(16) << "Name"       << ": " << name << endl;
        cout << left << setw(16) << "Department" << ": " << department << endl;
        cout << left << setw(16) << "Grade"      << ": " << grade << endl;
        cout << left << setw(16) << "Status"     << ": " << (isActive ? "Active" : "Inactive") << endl;
        cout << dash << endl;
        cout << left << setw(16) << "Basic Salary"   << ": Rs. " << formatMoney(basicSalary) << endl;
        cout << left << setw(16) << allowLabel.str()  << ": Rs. " << formatMoney(computeAllowances()) <<
endl;
        cout << left << setw(16) << "Gross Salary"    << ": Rs. " << formatMoney(computeGrossSalary()) <<
endl;
        cout << dash << endl;
        cout << left << setw(16) << "Tax Deduction"   << ": Rs. " << formatMoney(computeTax()) << endl;
        cout << left << setw(16) << "Net Salary"       << ": Rs. " << formatMoney(computeNetSalary()) <<
endl;
        cout << border << endl << endl;
    }

    static int getEmployeeCount() {
        return employeeCount;
    }

    void acceptDetails() {
        string n, dept, gradeStr, salaryStr, statusStr;

        cout << "Emp ID: " << empId << endl;
```

```
        cout << "Enter name: ";
        getline(cin, n);
        setName(n);

        cout << "Enter department: ";
        getline(cin, dept);
        setDepartment(dept);

        cout << "Enter grade: ";
        getline(cin, gradeStr);
        char g = gradeStr.empty() ? ' ' : gradeStr[0];
        setGrade(g);

        cout << "Enter basic salary: ";
        getline(cin, salaryStr);
        double salary = stod(salaryStr);
        setBasicSalary(salary);

        cout << "Enter status (Active/Inactive): ";
        getline(cin, statusStr);
        if (statusStr == "Inactive") {
            deactivate();
        } else if (statusStr != "Active") {
            cout << "ERROR: Status must be 'Active' or 'Inactive'. Defaulting to Active." << endl;
        }
    }
};

int Employee::employeeCount = 0;
int Employee::nextEmpId = 1001;

int main() {
    Employee e1;
    Employee* e2 = new Employee();
    Employee* e3 = new Employee();

    e1.acceptDetails();
    e2->acceptDetails();
    e3->acceptDetails();

    e1.printPayslip();
    e2->printPayslip();
    e3->printPayslip();
```

```
    e3->deactivate();
    if (!e3->getIsActive()) {
        cout << e3->getName() << " is no longer active. Payroll skipped." << endl;
    }

    cout << "Total Employees : " << Employee::getEmployeeCount() << endl;

    delete e2;
    delete e3;

    return 0;
}
```

Expected Output:

```
==============================================
        EMPLOYEE PAYSLIP - AUG 2026
==============================================
Emp ID          : 1001
Name            : Priya Sharma
Department      : Engineering
Grade           : B
Status          : Active
----------------------------------------------
Basic Salary    : Rs. 75,000.00
Allowances (30%): Rs. 22,500.00
Gross Salary    : Rs. 97,500.00
----------------------------------------------
Tax Deduction   : Rs. 9,750.00
Net Salary      : Rs. 87,750.00
==============================================
```

Bonus — Struct Padding (Optional, 5 marks)

During a code review, a colleague asks why two structs with the same members but different order have different sizes. You need to explain it.

struct Layout1 { char c1; int i; char c2; };
struct Layout2 { int i; char c1; char c2; };

Print sizeof(Layout1) and sizeof(Layout2). Inside a comment block, explain:
1.Why the sizes differ
2.What padding is and why the compiler adds it
3.Why member order matters when defining network packet headers or hardware register maps

Code:

```
#include <iostream>
using namespace std;

struct Layout1 {
   char c1;
   int  i;
   char c2;
};

struct Layout2 {
   int  i;
   char c1;
   char c2;
};

int main() {
   cout << "sizeof(Layout1) = " << sizeof(Layout1) << " bytes" << endl;
   cout << "sizeof(Layout2) = " << sizeof(Layout2) << " bytes" << endl;
   return 0;
}
```

Output:

```
sizeof(Layout1) = 12 bytes
sizeof(Layout2) = 8 bytes
```